

CS 486/586

Chapter 4 Understanding Data Types

Credit and thank you go to Professor R.Jesse Chaney from Portland State University for generously providing me with this material.

Unicode

Unicode is a computing industry standard for the consistent encoding, representation, and handling of text expressed in **most** of the world's writing systems.

Developed in conjunction with the **Universal Coded Character Set** (UCS) standard and published as **The Unicode Standard**, the latest version of Unicode contains a repertoire of more than **128,000 characters** covering **135 modern and historic scripts**, as well as multiple symbol sets.

Ligatures

In writing and typography, a **ligature** occurs where **two or more graphemes or letters are joined to form a single glyph**.

Examples are the characters ⟨æ⟩ and ⟨œ⟩ used in English and French, in which the letters ⟨a⟩ and ⟨e⟩ are joined for the first ligature and the letters ⟨o⟩ and ⟨e⟩ are joined for the second ligature.

The common ampersand, ⟨&⟩, developed from a ligature in which the handwritten Latin letters ⟨e⟩ and ⟨t⟩ (spelling et, Latin for 'and') were combined.

Unicode, cont 1

- **Unicode** can be implemented by different character encodings.
- The most commonly used encodings are **UTF-8** and **UTF-16**.
- **UTF-8** (8-bit Unicode Transformation Format) uses one byte for any **ASCII** character, all of which have the same code values in both UTF-8 and ASCII encoding, and up to four bytes for other characters.
- **UTF-16** uses one **16-bit** unit for the characters that were representable in UCS-2 and two 16-bit units (4×8 bits) to handle each of the additional characters.

Unicode, cont 2

Ligatures

- Many scripts, including Arabic and Devanagari, have special orthographic rules that require certain combinations of letterforms to be combined into **special ligature forms**.
- A ligature occurs where two or more graphemes or letters are **joined as a single glyph**.
- An example is **the character æ** as used in Danish, Norwegian, or Icelandic languages, or Old English (such as **dæmon**).
- The **German Eszett ß** is an official letter of the alphabet in Germany and Austria.
- **The common ampersand (&) developed from a ligature** in which the handwritten Latin letters e and t (spelling et, from the Latin for "and") were combined.

æ

&

Ligature Examples

Chandas

अआ इ ई उ ऊ ॐ
ऋ ॠ लृ ए ऐ
ओ औ तत् त्वम् असि

क ख ग घ ङ च छ ज झ ञ ट
ठ ड ढ ण त थ द ध न प फ
ब भ म य र ल ळ व श ष स ह

Devanagari script (vowels top,
consonants bottom) in Chandas font.

Arabic alphabet

خ	ح	ج	ث	ت	ب	أ
kha'	h'aa'	jiim	thaa'	taa'	baa'	alif
ص	ش	س	ز	ر	ذ	د
saad	shiin	siin	zaay	raa'	thaal	daal
ق	ف	غ	ع	ظ	ط	ض
qaaf	feh'	ghayn	'ayn	thaa	Taa'	daad
ي	و	ه	ن	م	ل	ك
yaa'	waaw	haa	nuun	miim	laam	kaaf

Basic Arabic alphabet

Diacritical Marks

é - accent acute
 è - accent grave
 ê - circumflex
 ë - umlaut or diaeresis
 ç - cedilla
 ñ - tilde
 ø - streg
 ð - eth (capital form Ð)
 å - bolle
 ē - macron
 č - háček
 ů - crescent

Characters with diacritical marks can generally be represented either as a single pre-composed character or as a decomposed sequence of a base letter plus one or more non-spacing marks.

Languages such as Czech make **heavy** use of diacritical marks.

A	a	I	i	S	s
Á	á	Í	í	Š	š
B	b	J	j	T	t
C	c	K	k	Ť	ť
Č	č	L	l	U	u
D	d	M	m	Ú	ů
Ď	ď	N	n	Ů	ů
E	e	Ň	ň	V	v
É	é	O	o	W	w
Ě	ě	Ó	ó	X	x
F	f	P	p	Y	y
G	g	Q	q	Ý	ý
H	h	R	r	Z	z
Ch	ch	Ř	ř	Ž	ž

Language with the highest ratio

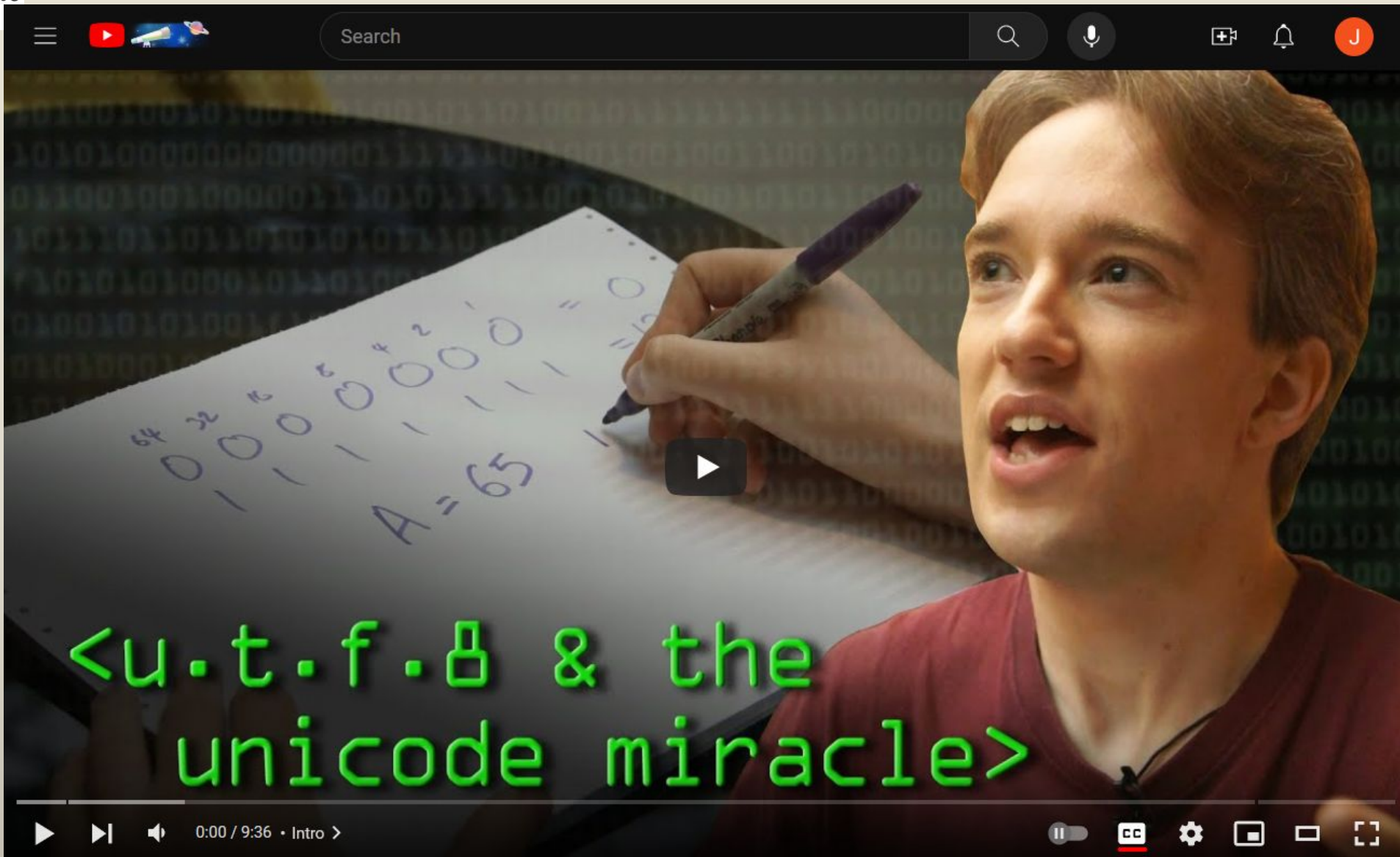
The Vietnamese alphabet

The official Latin-based Vietnamese alphabet

Aa	Ăă	Ââ	Bb	Cc	Dd	Đđ	Ee	Êê	Gg	Hh	Ii	Kk	Ll	Mm	Nn
Oo	Ôô	Ơơ	Pp	Qq	Rr	Ss	Tt	Uu	Ưư	Vv	Xx	Yy			

Vietnamese has an extensive number of letters with diacritical marks to make tonal distinctions.

Áá Àà Ăă Ãã Ạạ Ằằ Ẵẵ Ẳẳ Ẵẵ Ạạ Ââ Ấấ Ầầ Ẫẫ
 Ẫẫ Ậậ Đđ Éé Èè Ẻẻ Ẽẽ Ệệ Êê ẾếỀề Ễễ Ễễ Ệệ íì ìì
 ỉỉ ỉỉ ịị Óó Òò Ổổ Ỗỗ Ọọ Ôô ỐốỒồ Ỗỗ Ỗỗ Ỗỗ Ớớ
 Ớớ Ờờ Ởở Ỡỡ Ợợ Úú Ùù Ủủ Ỡỡ Ỡỡ Ỡỡ Ỡỡ Ỡỡ
 Ỡỡ Ỡỡ Ỡỡ Ýý Òò Ỡỡ Ỡỡ Ỡỡ Ỡỡ



<https://www.youtube.com/watch?v=MijmeoH9LT4>

Types of Data Types

PostgreSQL data types ***generally*** fall into the three categories you'll encounter most:

1. **Characters:** Any character or symbol
2. **Numbers:** Includes whole numbers and fractions
3. **Dates and times:** Temporal information

Some of the data types are ANSI, some are not.

A **more** complete list of available data types in PostgreSQL is:

Character	Network address
Numeric	Arrays
Date and time	Range
Boolean	Geometric
Bit string	XML
Enumerated	JSON
UUID	Monetary
Large Object (LOB	Others...

The Character Data Types

Character string types are general-purpose types suitable for any combination of text, numbers, and symbols. Character types include the following:

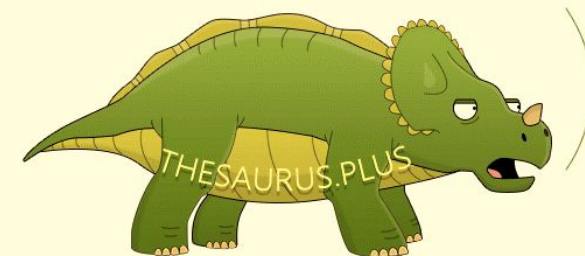
- **char(n)**: A fixed-length column where the character length is specified by n. This always stores n number of characters in the column.
- **varchar(n)**: A variable-length column where the maximum length is specified by n. This can store less than the max number of characters in the column.
- **text**: A variable-length column of unlimited length. Based to the PostgreSQL documentation, the longest possible character string you can store is about 1 gigabyte. **Non-ANSI**

<https://www.postgresql.org/docs/current/datatype-character.html>

char (n)

- A **fixed-length column** where the character length is specified by n.
- A column of **char(20)** **always stores 20 characters** per row regardless of how many characters you insert.
- **If you insert fewer than 20 characters in any row, PostgreSQL pads the rest of that column with spaces.**
- This type, which is **part of standard SQL**, also can be specified with the longer name **character(n)**.
- If specified, the length n must be greater than zero and **cannot exceed 10,485,760**.
- A column of **char (n)** **is used infrequently** and is **mainly a remnant of legacy computer systems**.

synonyms for remnant:
remainder, residue, rest, remains, end, leftover,
balance, vestige, residuum, piece



- Values of type **char** are physically padded with spaces to the specified width **n**, and are stored and displayed that way.
- **However**, trailing spaces are treated as semantically insignificant and disregarded when comparing two values of type **char**.
- In collations where whitespace is significant, this behavior can produce unexpected results; for example `SELECT 'a'::CHAR(2) collate "C" < E'a\n'::CHAR(2)` returns true, even though C locale would consider a space to be greater than a newline.
- Trailing spaces are removed when converting a **char** value to one of the other string types.
- Note that trailing spaces are semantically significant in **character varying** and **text** values, and when using pattern matching, that is **LIKE** and regular expressions.

varchar (n)

- A variable-length column where the maximum length is specified by n.
- If you insert fewer characters than the maximum, PostgreSQL will not store extra spaces.
 - For example, the string 'blue' will take four spaces, whereas the string '123' will take three.
- In large databases, this practice saves considerable space.
- This type, included in standard SQL, also can be specified using the longer name **character varying (n)**.
- If specified, the length **n** must be greater than zero and cannot exceed 10,485,760.
- If **varchar** is used without length specifier, the type accepts strings of "any length."

text

- A **variable-length column of “unlimited” length**.
- According to the PostgreSQL documentation, the longest possible character string you can store is about 1 gigabyte.
 - “In any case, the longest possible character string that can be stored is about 1 GB.”
- The text type is **not part of the SQL standard**, but you’ll find similar implementations in other database systems, including Microsoft SQL Server and MySQL.

Performance Difference Among the Three Character Types

- There is **no performance difference among these three types**, apart from increased storage space when using the blank-padded type, and a few extra CPU cycles to check the length when storing into a length-constrained column.
- While `char (n)` has performance advantages in some other database systems, there is no such advantage in PostgreSQL; **in fact `char (n)` is usually the slowest** of the three because of its additional storage costs.
- **In most situations, `text` or character `varchar` should be used.**

<https://www.postgresql.org/docs/current/datatype-character.html>

```
CREATE TABLE data_types_examples.char_data_types (  
    char_column char(10)  
    , varchar_column varchar(10)  
    , text_column text  
);
```

```
INSERT INTO data_types_examples.char_data_types  
VALUES  
    ('abc', 'abc', 'abc')  
    , ('defghi', 'defghi', 'defghi');
```

```
select
  char_column as "char"
  , varchar_column as "varchar"
  , text_column as "text"
from data_types_examples.char_data_types;
```

char	varchar	text
abc	abc	abc
defghi	defghi	defghi

You cannot perform arithmetic operations on numbers stored in a character column.

Store numeric value as character types only when they represent codes, such as a US postal ZIP code.

The Numeric Data Types

The numeric data types allow you to perform arithmetic calculations on those numbers.

The SQL number types include the following:

- **Integers:** Whole numbers, both positive and negative.
 - As you'd expect, values of this type cannot represent a value with a fractional component.
 - There is no separate representation for unsigned integers.
- **Fixed-point and floating-point:** Two formats of fractions of values that can represent a fractional component.

The Integer Types

The SQL standard provides three integer types:

1. **smallint**,
2. **integer**, and
3. **bigint**.

The difference between the three types is the **maximum size of the numbers they can hold**.

Name	Storage Size	Description	Range
<code>smallint</code>	2 bytes	small-range integer	-32768 to +32767
<code>integer</code>	4 bytes	typical choice for integer	-2147483648 to +2147483647
<code>bigint</code>	8 bytes	large-range integer	-9223372036854775808 to +9223372036854775807

The Auto-Incrementing Integer Types

The data types

smallserial,
serial and
bigserial

I strongly discourage you from using these because there is an ANSI alternative.

are not true types, but merely **a notational convenience** for creating unique identifier columns. **These are not ANSI SQL.**

It is often helpful to create a column that holds **integers that autoincrement each time you add a row to the table.**

For example, you might use an auto-incrementing column to create a unique ID number, also known as a primary key, for each row in the table.

The Decimal Types

The decimal types are broken into 2 separate categories:

1. **Floating point**: this is what you normally think of when you compare it to a programming language type such as float in C.
2. **Arbitrary Precision Numbers** (aka **Fixed Point Numbers**): The type numeric can store numbers with a very large number of digits. **It is especially recommended for storing monetary amounts and other quantities where exactness is required.**

Floating-Point Types

- The two floating-point types are **real** and **double precision**, both part of the **SQL standard**.
- The difference between the two is how much data they store.
- The `real` type allows precision to **six decimal digits**, and `double precision` to **15 decimal digits of precision**, both of which include the number of digits on both sides of the point.
- On all currently supported platforms, these types are implementations of **IEEE Standard 754** for Binary Floating-Point Arithmetic (single and double precision, respectively), to the extent that the underlying processor, operating system, and compiler support it.

Floating-Point Types

Name	Storage Size	Description	Range
real	4 bytes	variable-precision, inexact	6 decimal digits precision
double precision	8 bytes	variable-precision, inexact	15 decimal digits precision

Arbitrary Precision Types

- The type numeric can store numbers with a **very large number of digits**.
- It is especially **recommended for storing monetary amounts and other quantities where exactness is required**.
- Calculations with numeric values yield exact results where possible, e.g., addition, subtraction, multiplication.
- However, **calculations on numeric values are very slow compared to the integer types, or to the floating-point types**.

Arbitrary Precision Types

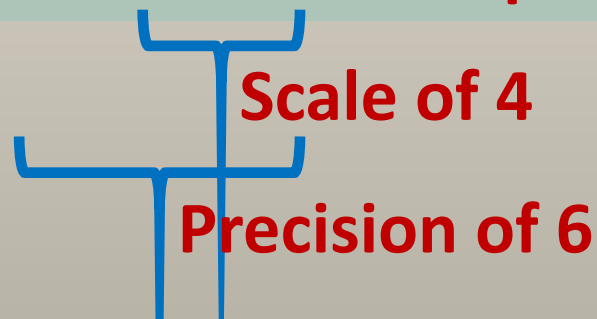
The **precision** of a numeric is the total count of significant digits in the entire number, the total number of digits to both sides of the decimal point.

The **scale** of a numeric is the count of decimal digits in the fractional part, to the right of the decimal point.

So, the number 23.5141 has a precision of 6 and a scale of 4.

We do not count the decimal point as part of the precision.

The number 23.5141 has a **precision of 6** and a **scale of 4**.



Arbitrary Precision Types

The number 23.5141 has a **precision of 6** and a **scale of 4**.

23.5141

scale of 4

precision of 6

23

scale of 0

precision of 2

Notice that **the decimal point is not counted** anywhere in either the precision or scale. This differs from how you specify a `float` or `double` in C with a format string.

Arbitrary Precision Types

Name	Storage Size	Description	Range
decimal	variable	user-specified precision, exact	up to 131072 digits before the decimal point; up to 16383 digits after the decimal point
numeric	Same as decimal	Same as decimal	Same as decimal

Arbitrary Precision Types

numeric (precision, scale)	Both the maximum precision and the maximum scale of a numeric column can be configured.
numeric (precision)	If you omit specifying a scale value, the scale will be set to zero; in effect, that creates an integer.
numeric	If you omit specifying the precision and the scale, the database will store values of any precision and scale up to the maximum allowed.

```
CREATE TABLE
```

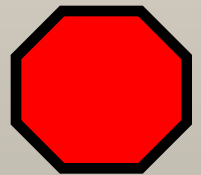
```
data_types_examples.number_data_types (  
    numeric_column numeric(20,5),  
    real_column    real,  
    double_column  double precision  
);
```

```
INSERT INTO data_types_examples.number_data_types  
VALUES  
    (.7, .7, .7),  
    (2.13579, 2.13579, 2.13579),  
    (2.1357987654, 2.1357987654, 2.1357987654);
```

```
SELECT *  
FROM data_types_examples.number_data_types
```

numeric_column	real_column	double_column
0.7	0.7	0.7
2.13579	2.13579	2.13579
2.13580	2.1357987	2.1357987654

scale of 5



Date and Time Types

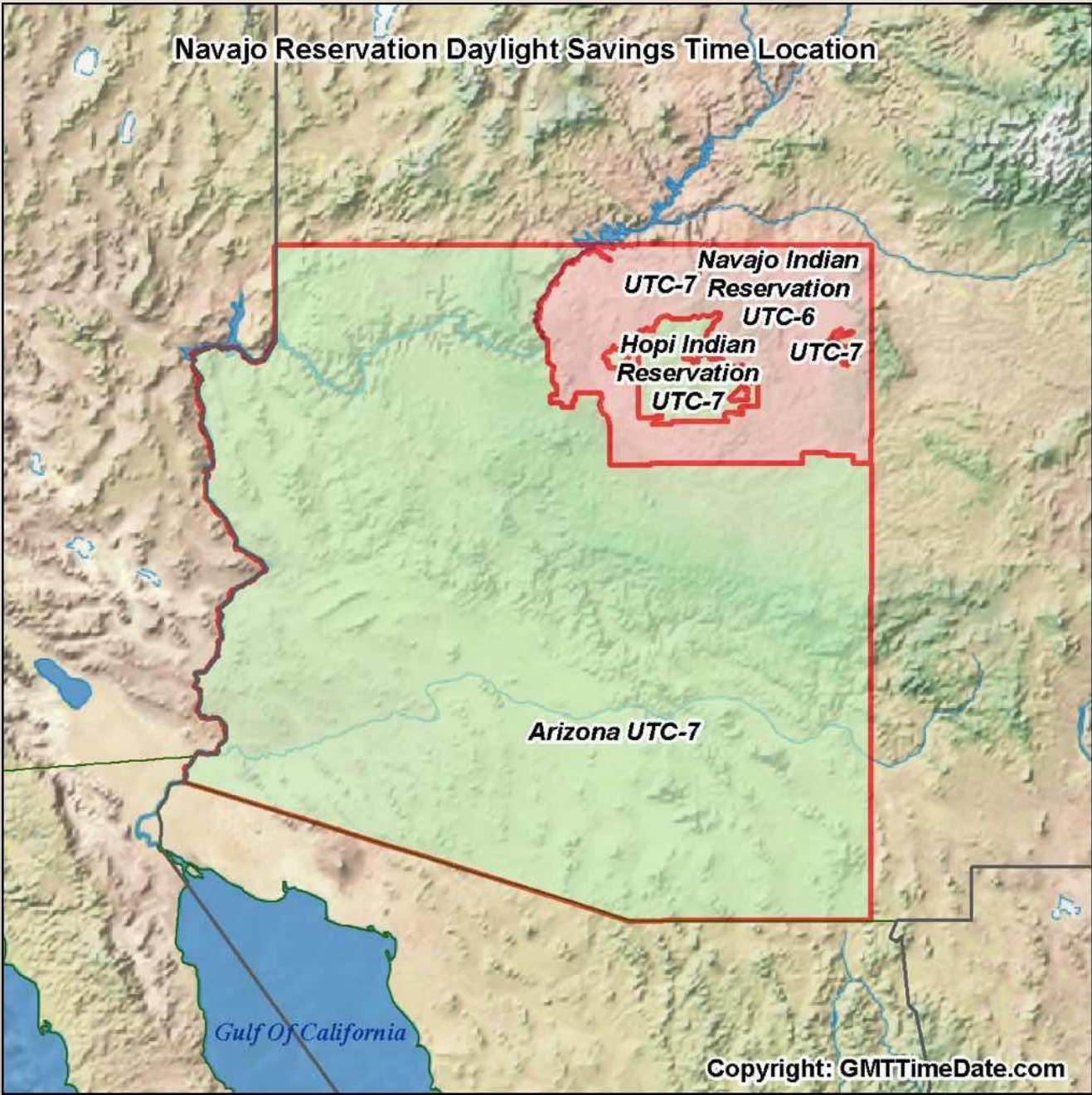
Whenever you enter a date into a search form, you're reaping the benefit of databases having an awareness of the current time (received from the server) plus the ability to handle formats for dates, times, and the **nuances of the calendar**, such as **leap years and time zones**.

Dates are counted according to the Gregorian calendar, even in years before that calendar was introduced.



<https://www.youtube.com/watch?v=-5wpm-gesOY>

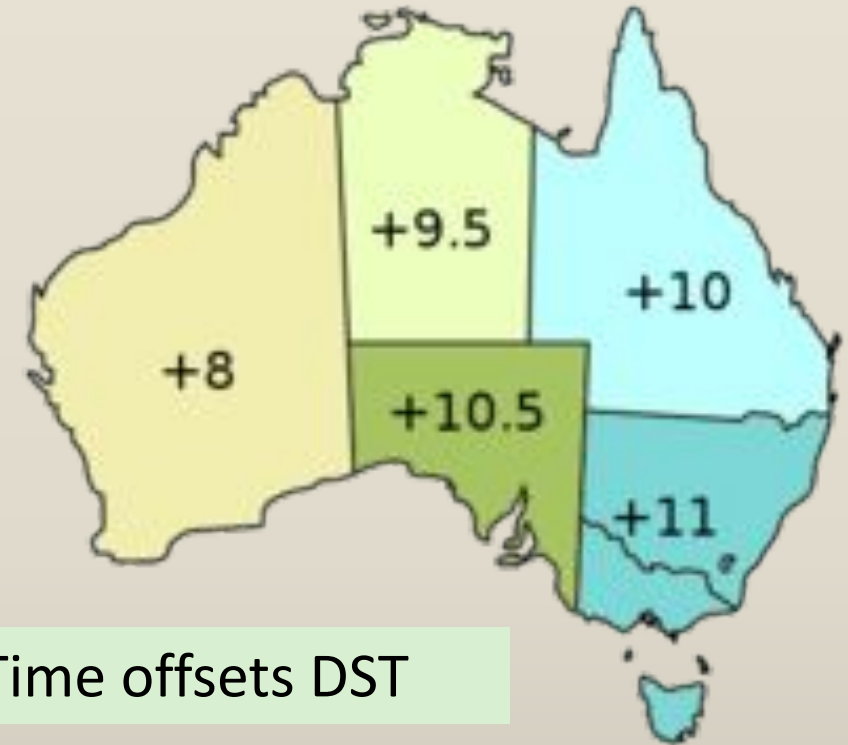
Some Time Zones



Some Time Zones



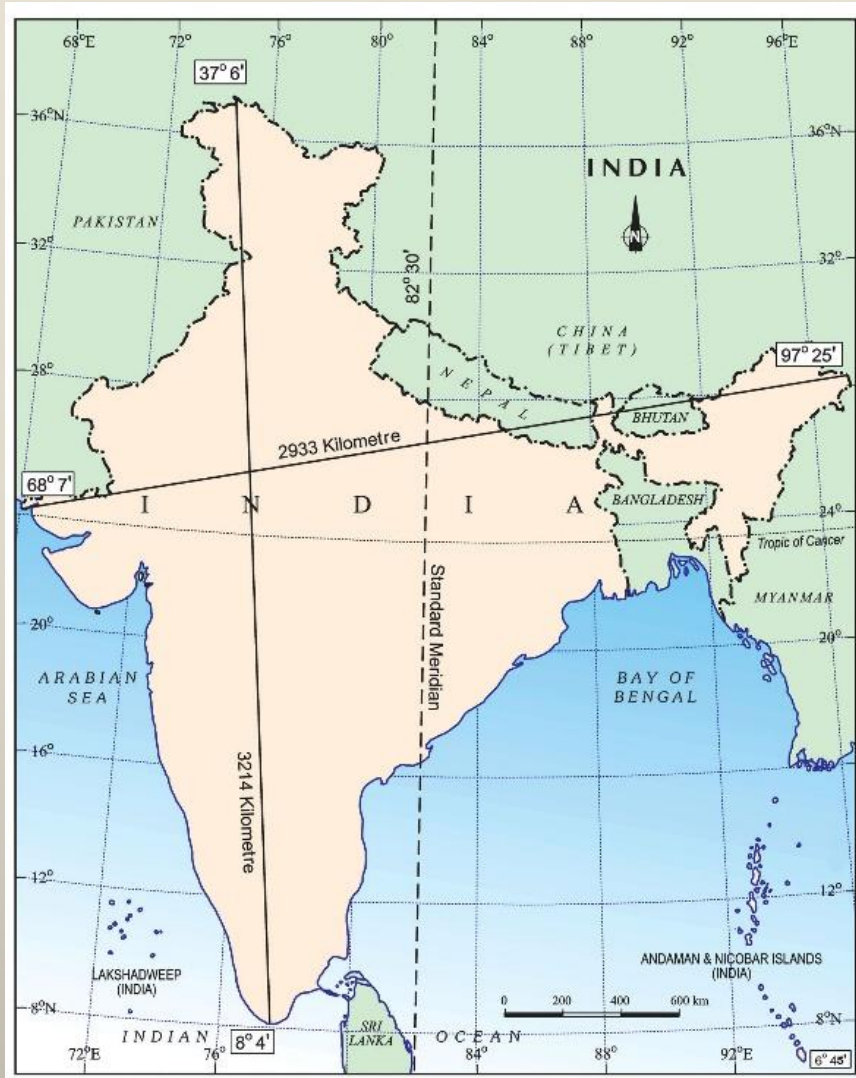
Time offsets for
standard time



Time offsets DST

A compromise between Western and Central time (**UTC+8:45**, without DST), **unofficially known as Central Western Standard Time**, is used in one area in the southeastern corner of Western Australia **and one roadhouse in South Australia.**

Some Time Zones



IST – India Standard Time (IST) is **5:30 hours** ahead of **Coordinated Universal Time**.

Nepal Standard Time (**UTC+05:45**) DST not observed.



babbage ~
rchaney # cal 9 1752
September 1752
Su Mo Tu We Th Fr Sa
 1 2 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30

“September 2, 1752 was when **Great Britain and her colonies switched to the Gregorian Calendar.**”

Name	Storage Size	Description	Low Value	High Value	Resolution
timestamp [(p)] [without time zone]	8 bytes	both date and time (no time zone)	4713 BCE	294276 CE	1 microsecond
timestamp [(p)] with time zone	8 bytes	both date and time, with time zone	4713 BCE	294276 CE	1 microsecond
date	4 bytes	date (no time of day)	4713 BCE	5874897 CE	1 day
time [(p)] [without time zone]	8 bytes	time of day (no date)	00:00:00	24:00:00	1 microsecond
time [(p)] with time zone	12 bytes	time of day (no date), with time zone	00:00:00+1559	24:00:00-1559	1 microsecond
interval [<i>fields</i>] [(p)]	16 bytes	time interval	Common Era (CE) and Before the Common Era (BCE) 0 years years		

timestamp

- Records **date and time**, which are useful for a range of situations you might track: departures and arrivals of passenger flights, a schedule of sporting events, or incidents along a timeline.
- **You will almost always want to add the keywords with time zone** to ensure that the time recorded for an event includes the time zone where it occurred.
- Otherwise, times recorded in various places around the globe become impossible to compare.
- The format timestamp **with time zone** is part of the **SQL standard**.

date

- Records **just the date**. **Is part of the SQL standard.**

time

- Records just the time and **is part of the SQL standard.**
- Although you can add the `with time zone` keywords, **without a date the time zone will be meaningless.**

interval

- Holds a value representing a unit of time expressed in the format quantity unit.
- It doesn't record the start or end of a time period, only its duration. Examples include 12 days or 8 hours. **Is part of the SQL standard.**

```
CREATE TABLE if not exists
data_types_examples.date_time_types (
    timestamp_column timestamp with time zone,
    interval_column interval
);

INSERT INTO data_types_examples.date_time_types
VALUES
    ('2022-12-31 01:00 EST' , '2 days'),
    ('2022-12-31 01:00 -8' , '1 month'),
    ('2022-12-31 01:00 Australia/Melbourne' , '1 century'),
    (now() , '1 week');
```

```
SELECT
  timestamp_column
  , interval_column
  , timestamp_column - interval_column AS new_date
FROM data_types_examples.date_time_types;
```

timestamp_column	interval_column	new_date
2022-12-30 22:00:00.000-0800	2 days	2022-12-28 22:00:00.000-0800
2022-12-31 01:00:00.000-0800	1 mon	2022-11-30 01:00:00.000-0800
2022-12-30 06:00:00.000-0800	100 years	1922-12-30 06:00:00.000-0800
2023-10-05 05:43:19.371-0700	7 days	2023-09-28 05:43:19.371-0700

Understanding JSON and JSONB

JSON, short for **JavaScript Object Notation**, is a structured data format used for both storing data and exchanging data between computer systems.

All major programming languages support reading and writing data in JSON format, which organizes information in a **collection of key/value pairs as well as lists of values**.

Understanding JSON and JSONB

- The **json** and **jsonb** data types accept **almost identical** sets of values as input.
- **The major practical difference is one of efficiency.**
- The **json** data type **stores an exact copy** of the input text, which processing functions must reparse on each execution.
- The **jsonb** data is stored in a **decomposed binary format** that makes it **slightly slower to input** due to added conversion overhead, but **significantly faster to process**, since no reparsing is needed.
- **jsonb** also **supports indexing**, which can be a significant advantage.
- **Most applications should prefer to store JSON data as jsonb**, unless there are quite specialized needs.


```
CREATE TABLE data_types_examples.orders (  
    id integer  
        generated by default as identity primary key  
    , info json NOT NULL  
);
```

```
INSERT INTO data_types_examples.orders (info)  
VALUES  
    ('{ "customer": "Lily Bush", "items":  
        {"product": "Diaper", "qty": 24}}')  
    , ('{ "customer": "Josh William", "items":  
        {"product": "Toy Car", "qty": 1}}')  
    , ('{ "customer": "Mary Clark", "items":  
        {"product": "Toy Train", "qty": 2}}');
```

```
select *  
from data_types_examples.orders;
```

id	info
1	{ "customer": "Lily Bush", "items": {"product": "Diaper","qty": 24}}
2	{ "customer": "Josh William", "items": {"product": "Toy Car","qty": 1}}
3	{ "customer": "Mary Clark", "items": {"product": "Toy Train","qty": 2}}

```
SELECT info -> 'customer' AS customer  
FROM data_types_examples.orders;
```

customer
"Lily Bush"
"Josh William"
"Mary Clark"

```
SELECT
  info -> 'customer' AS customer
, info -> 'items' -> 'qty' AS qty
FROM data_types_examples.orders;
```

customer	qty
"Lily Bush"	24
"Josh William"	1
"Mary Clark"	2

Boolean Type

- PostgreSQL provides the **standard SQL type boolean**.
- The **boolean** type can have **three** states:
 - “true”,
 - “false”, and
 - a **third state**, “unknown”, which is represented by the SQL **NULL** value.



Transforming Values from One Type to Another with **CAST**

Occasionally, we need to **transform a value from its stored data type to another type**. For example, you want to retrieve a number as a character so you can combine it with text. Or you might need to convert dates stored as characters into an actual date type so you can sort them in date order or perform interval calculations. You can perform these conversions using the **CAST ()** function.

CAST from One Type to Another

```
SELECT
  timestamp_column
  , CAST( timestamp_column AS varchar(10) )
FROM data_types_examples.date_time_types;
```

timestamp_column	timestamp_column
2022-12-30 22:00:00.000 -0800	2022-12-30
2022-12-31 01:00:00.000 -0800	2022-12-31
2022-12-30 06:00:00.000 -0800	2022-12-30
2025-01-12 22:15:48.583 -0800	2025-01-12

CAST from One Type to Another

```
SELECT numeric_column,  
       CAST(numeric_column AS integer),  
       CAST(numeric_column AS text)  
FROM data_types_examples.number_data_types;
```

numeric_column	numeric_column	numeric_column
0.7	1	0.70000
2.13579	2	2.13579
2.1358	2	2.13580

Not all **CAST** Operations Work

-- Does not work

SELECT

CAST(char_column **AS integer**)

FROM data_types_examples.char_data_types;

Using the **CAST** Shortcut Notation

```
SELECT
    timestamp_column
    , CAST(timestamp_column AS varchar(10))
FROM date_time_types;
```

```
SELECT
    timestamp_column::varchar(10)
FROM date_time_types;
```